

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Technical Presentation (Low)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Prepare a 5-slide presentation introducing DBMS: definition, objectives, advantages, and real-world applications.	BL2 – Understand	5
2	Present a comparison between file-based systems and DBMS using real-life case studies from banking or healthcare.	BL2 – Understand	5
3	Deliver a presentation explaining the three-level ANSI/SPARC architecture using diagrams and practical examples.	BL2 – Understand	5
4	Present the different data models (Hierarchical, Network, Relational) with diagrams and industry use cases.	BL1 – Remember	5
5	Create and present an ER diagram for an Online Library System, explaining each component clearly.	BL2 – Understand	5
6	Prepare a presentation on the EER model, demonstrating generalization and specialization with a University case study.	BL2 – Understand	5
7	Present the concept of data independence (logical and physical) and its significance in DBMS architecture.	BL2 – Understand	5

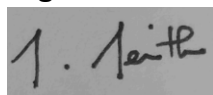
8	Deliver a technical presentation on the software components of a DBMS, explaining query processor, storage manager, and transaction manager.	BL1 – Remember	5
9	Prepare a presentation comparing strong and weak entities in ER modeling with real-world examples.	BL2 – Understand	5
10	Present a complete design walkthrough of an EER diagram for a Retail Chain Management System.	BL2 – Understand	5

Code of Conduct:

I jenitha jothi 192520013 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach

Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

What is a DBMS?

A **Database Management System (DBMS)** is a specialized software layer that interacts with end users, applications, and the physical database to capture and analyze data efficiently.

- Acts as a secure intermediary between users and raw data files.
- Provides standard interface languages (e.g., SQL, DDL, DML).
- Manages physical storage, indexing, and memory buffers.

Primary Objectives

- **Data Persistence:** Ensures durable data storage across system power cycles.
- **Data Abstraction:** Hides complex physical storage structures from application code.
- **Shared Access:** Enables thousands of concurrent users to access data simultaneously.
- **Integrity Enforcement:** Automatically validates constraints and data accuracy rules.

What is a DBMS?

A **Database Management System (DBMS)** is a specialized software layer that interacts with end users, applications, and the physical database to capture and analyze data efficiently.

- Acts as a secure intermediary between users and raw data files.
- Provides standard interface languages (e.g., SQL, DDL, DML).
- Manages physical storage, indexing, and memory buffers.

Primary Objectives

- **Data Persistence:** Ensures durable data storage across system power cycles.
- **Data Abstraction:** Hides complex physical storage structures from application code.
- **Shared Access:** Enables thousands of concurrent users to access data simultaneously.
- **Integrity Enforcement:** Automatically validates constraints and data accuracy rules.



ACID Guarantees

Enforces Atomicity, Consistency, Isolation, and Durability to prevent corruption during system crashes or power failures.



Reduced Redundancy

Centralized database schemas eliminate duplicate file copies, optimizing disk space and preventing data drift.



Industry Use Cases

Banking: Account balance transactions.
E-Commerce: Order inventory tracking.
Healthcare: EHR patient treatment histories.

📁 File-Based Storage Architecture

In traditional file systems, each application program maintains its own isolated data files (e.g., CSV, COBOL files, Excel sheets).

- Data formats are tightly coupled to application code.
- Changes in file format require rewriting application code.
- No central security model or coordinated access protocol.

⚠️ Severe Vulnerabilities

- **Data Inconsistency:** Updating a customer address in one file leaves stale copies in others.
- **Concurrency Anomalies:** Simultaneous file edits cause partial writes or file corruption.
- **Data Isolation:** Querying data across departmental files requires custom code.

Parameter	File-Based Processing System	Database Management System (DBMS)
Data Redundancy	High; duplicate data files in every department	Controlled & minimal through normalization
Data Independence	Absent; tight coupling between files & programs	High; physical and logical layers separated
Concurrency Control	None or coarse-grained OS file locking	Granular row/page lock engines & ACID transactions
Data Security	Basic OS directory/file read-write permissions	Role-based access control (RBAC) & encryption
Crash Recovery	Manual backup restoration; potential data loss	Automated Write-Ahead Logging (WAL)

🏦 Banking: File-System Failure

A customer withdraws \$200 at an ATM while a auto-debit of \$200 processes simultaneously. Without transaction isolation, both reads observe a \$300 balance, leaving a final negative balance due to lost update anomalies.

✅ DBMS Banking Solution

DBMS uses strict two-phase locking (2PL). ATM read locks the account row; auto-debit waits in queue until ATM transaction commits, maintaining accurate balances.

🏥 Healthcare: File-System Hazard

A patient's severe penicillin allergy is logged in an Outpatient CSV file, but absent in Emergency Ward files. Doctors administer lethal medication during emergency care.

✅ DBMS Healthcare Solution

Unified relational database with Patient_ID foreign keys gives instant, real-time access to complete clinical history across all medical wards.

The **ANSI/SPARC 3-Level Architecture** divides database management into three discrete abstraction layers to decouple user applications from raw physical hardware details.

- **External Level:** Customized user views.
- **Conceptual Level:** Global logical schema design.
- **Internal Level:** Physical storage and indexing layout.

Key Goal

Ensure that changes at low physical storage levels do not break user applications or top-level reporting interfaces.

External View 1 | External View 2



Conceptual Schema (Logical Entities)



Internal Schema (B-Trees & Page Blocks)



1. External Level

The top layer providing tailored user views (e.g., SQL Views, UI forms). Hides irrelevant table attributes from specific departments to streamline UX and enforce privacy.



2. Conceptual Level

Describes *what* data is stored across the entire enterprise. Contains entity definitions, attributes, relationships, security policies, and integrity constraints.



3. Internal Level

Describes *how* data is stored physically on disk. Specifies record formats, B+ tree indexing structures, block sizes, allocation strategies, and compression algorithms.

🔗 External / Conceptual Mapping

Maps elements of user views to fields in the conceptual schema.

- **View Query:** SELECT Name, Balance FROM Accounts;
- **Conceptual Table:** Accounts(AccNo, Name, SSN, Balance, BranchID)
- Allows virtual field transformations (e.g., computing age from DOB).

⚙️ Conceptual / Internal Mapping

Maps conceptual fields to physical storage blocks and file layouts.

- Specifies that AccNo is stored as a 4-byte integer.
- Defines a B+ tree index on BranchID stored in file block 0x0F4A.
- Translates logical SQL queries into physical disk block offsets.

A **Data Model** is an abstract framework providing concepts and notations to describe data structures, relationships, semantics, and operational constraints.

- **Schema:** Overall design/structure of the database (rarely changes).
- **Instance:** Collection of actual data stored at a specific moment in time.
- Acts as the architectural blueprint prior to database implementation.

 **Hierarchical Model (Tree)** **Network Model (Graph)** **Relational Model (Tables)**

Hierarchical Model

Organizes data into a tree structure with parent-child relationships (1:N). Fast navigation but rigid; cannot naturally represent Many-to-Many relationships without duplicate nodes.



Network Model

Uses a graph structure where child nodes can have multiple parent nodes (M:N via sets). Navigated via complex physical pointers; difficult to maintain and query flexible views.



Relational Model

Organizes data into 2D tables (relations) connected by foreign keys. Supports SQL, high data independence, declarative queries, and easy structural modifications.

Model Type	Data Structure	Relationship Support	Industry Use Case
Hierarchical	Inverted Tree Hierarchy	Strict 1:N (Single Parent)	IBM IMS, File Systems, XML Document Parsing
Network	Complex Graph / Pointers	1:N and M:N (Multiple Parents)	Legacy Telecom billing, CODASYL DB systems
Relational	Two-Dimensional Tables	1:1, 1:N, M:N via Junction Tables	PostgreSQL, Oracle DB, Banking, Enterprise ERP

Primary Entities

- **BOOK:** ISBN (PK), Title, Publisher, Year, Category.
- **MEMBER:** MemberID (PK), Name, Email, Phone, MembershipType.
- **AUTHOR:** AuthorID (PK), Name, Biography.
- **PUBLISHER:** PubID (PK), Name, Address.

Transactional / Weak Entities

- **LOAN (Transaction):** LoanID (PK), IssueDate, DueDate, ReturnDate.
- **FINE (Weak Entity):** FineID (Partial Key), FineAmount, PaidStatus. Dependent on LOAN.
- **BOOK COPY:** CopyNo, Status (Available/Issued). Dependent on BOOK.



WRITTEN_BY (M:N)

Connects **BOOK** and **AUTHOR**. One book can have multiple authors; one author can write multiple books.



BORROWS (1:N)

Connects **MEMBER** to **LOAN**. A member can initiate multiple loans; each loan belongs to exactly one member.



INCURS (1:1)

Connects **LOAN** to **FINE**. An overdue loan generates a fine record with total participation from FINE.



Notation Legend & Mapping

- **Rectangles:** Entity Sets (Book, Member, Author).
- **Double Rectangles:** Weak Entity (Fine).
- **Diamonds:** Relationship Sets (Borrows, Incurs).
- **Ovals:** Attributes; Underlined ovals are Primary Keys.

What is EER Modeling?

Enhanced Entity-Relationship (EER) modeling extends standard ER concepts with object-oriented semantics to handle complex domain logic.

- **Superclass:** A generic entity set containing shared attributes.
- **Subclass:** Specialized entity groupings inheriting superclass attributes.
- **Type Inheritance:** Subclasses automatically inherit all attributes and relationships of their superclass.

Key Abstraction Mechanisms

- **Specialization:** Top-down process of creating subclasses from a superclass.
- **Generalization:** Bottom-up process of synthesizing common subclasses into a superclass.
- **Categorization (UNION Types):** Representing a single subclass with multiple distinct superclass types.



Disjoint Constraint (d)

Specifies that an entity instance can belong to at most *one* subclass. (e.g., A Student cannot simultaneously be a Full-Time Faculty member).



Overlapping Constraint (o)

Specifies that an entity instance can simultaneously belong to *multiple* subclasses (e.g., A person can be both a Student and a Teaching Assistant).



Completeness (Total vs Partial)

Total (Double Line): Every superclass entity must belong to a subclass.

Partial (Single Line): Superclass entity can exist without subclass membership.

PERSON (SSN, Name, Email, DOB)

✓[d, Total Specialization]

STUDENT (GPA)

FACULTY (Rank)

STAFF (Shift)

Specialization Breakdown

- **STUDENT Subclasses:** Specialized further into UNDERGRAD (SAT score) and POSTGRAD (Thesis Topic).
- **FACULTY Subclasses:** Specialized into PROFESSOR and LECTURER.
- **Inheritance Example:** A POSTGRAD inherits SSN, Name, Email from PERSON, plus GPA from STUDENT.

QUESTION 7 Concept of Data Independence

Slide 19 / 30

🔒 Definition

Data Independence is the ability to modify the schema at one level of a database architecture without requiring modifications to the schema at the next higher level.

- Crucial feature enabled by the ANSI/SPARC 3-level architecture.
- Saves millions in software maintenance costs.
- Decouples user applications from physical storage upgrades.

📁 Two Categories

- **Physical Data Independence:** Immunity of conceptual schema to changes in physical storage structures.
- **Logical Data Independence:** Immunity of external schemas/user views to changes in the logical schema design.

QUESTION 7 Physical Data Independence

Slide 20 / 30

Refers to the capacity to change the **internal schema** without having to change the **conceptual schema** or application programs.

Typical Physical Storage Modifications

- Migrating database files from HDD to high-speed NVMe SSDs.
- Creating B+ tree or Hash indexes on frequently queried columns.
- Changing file organization (e.g., Sequential to Heap or Clustered).
- Compressing data pages or altering block allocation sizes.

Application SQL Query unchanged



Conceptual Schema: Accounts Table

✓ Physical Independence Shield

Physical Disk Storage / B-Tree Index Altered

QUESTION 7 Logical Data Independence & Significance

Slide 21 / 30

🔗 Logical Data Independence

Refers to changing the conceptual schema (adding/deleting tables or columns) without modifying existing external views or user queries.

- **Example:** Adding a new AadhaarNo column to the Customer table doesn't break an existing application query reading Name and Email.
- Handled through SQL Views that insulate older application queries.

★ Overall Significance

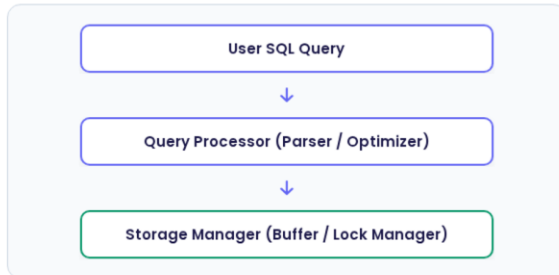
- Drastically lowers software maintenance and application refactoring overhead.
- Enables seamless database administrative tuning and hardware upgrades.
- Allows concurrent evolution of application features alongside underlying storage engines.

QUESTION 8 DBMS Software Components Architecture

Slide 22 / 30

A production DBMS engine is a highly complex software suite divided into two major operational subsystems:

- **Query Processor Subsystem:** Parses, optimizes, and compiles declarative SQL statements into low-level execution plans.
- **Storage Subsystem:** Manages memory buffers, disk allocation, lock concurrency, and transactional WAL logs.



QUESTION 8 The Query Processor Subsystem

Slide 23 / 30



1. DDL & DML Parser

Parses raw SQL text into syntax trees. Validates table/column names against the data dictionary (catalog) and enforces user authorization checks.



2. Query Optimizer

Generates multiple relational algebra execution plans. Evaluates statistics (histogram costs) to select the plan with the lowest estimated CPU and disk I/O cost.



3. Execution Engine

Executes the chosen low-level query plan by calling internal Storage Manager APIs to fetch and filter binary disk pages.

QUESTION 8 Storage Manager & Transaction Manager

Slide 24 / 30

Storage Manager Components

- **Buffer Manager:** Controls fetching disk blocks into RAM buffer cache using LRU eviction policies.
- **File Manager:** Allocates raw disk space and data structures on physical media.
- **Data Dictionary / Catalog:** Stores system metadata schema tables.

Transaction Manager Components

- **Lock Manager:** Grants shared/exclusive locks to enforce transaction isolation levels.
- **Log / Recovery Manager:** Writes undo/redo logs to guarantee durability and handle system crash recovery.

QUESTION 9 Strong vs. Weak Entities: Fundamentals

Slide 25 / 30

Strong Entity Set

An entity set that possesses a **primary key** capable of uniquely identifying each entity instance independently.

- Exists independently of other entities in the ER schema.
- Represented in ER diagrams by a *single rectangle*.
- Example:** EMPLOYEE(EmpID, Name, Salary).

Weak Entity Set

An entity set that lacks a primary key and is **existence-dependent** on a parent strong entity.

- Identified by combining parent PK with its own *partial key (discriminator)*.
- Represented in ER diagrams by a *double rectangle*.
- Example:** DEPENDENT(EmpID, DepName, Relation).

QUESTION 9 Strong vs. Weak Entities: Direct Comparison

Slide 26 / 30

Parameter	Strong Entity Set	Weak Entity Set
Primary Key	Has its own sufficient Primary Key	No PK; has a Partial Key (Discriminator)
Existence Dependency	Independent existence in database	Dependent on Owner Strong Entity
ER Diagram Symbol	Single Rectangle	Double Rectangle
Relationship Symbol	Single Diamond (Standard Relationship)	Double Diamond (Identifying Relationship)
Participation Constraint	Can be Partial or Total	Always Total Participation in identifying relationship

QUESTION 9 Real-World Examples & Relational Mapping

Slide 27 / 30

Real-World Case 1: Building & Room

A ROOM (Number 101) cannot exist without a BUILDING (Engineering Block). Room number 101 recurs in multiple buildings; hence BuildingID + RoomNo forms the composite primary key.

Real-World Case 2: Loan & Payment

A PAYMENT installment is uniquely identified only in the context of its parent bank LOAN record.

[BUILDING] BuildingID (Strong)

↓(Identifying Rel)

[[ROOM]] RoomNo (Weak)

**Store & Inventory**

Manages regional physical retail stores, warehouse stock counts, product categories, and price lists.

**Sales & POS Terminals**

Handles in-store Point-of-Sale checkout transactions and online web orders linked to customer accounts.

**Suppliers & Staff**

Tracks vendor purchase orders, supply shipments, store managers, and shift cashiers.

**PRODUCT Hierarchy**

Superclass PRODUCT (SKU, Name, BasePrice) specialized into:

- PERISHABLE_ITEM (ExpiryDate, StorageTemp Requirements).
- ELECTRONICS (WarrantyPeriod, Brand).
- APPAREL (Size, Color, FabricType).

**EMPLOYEE Hierarchy**

Superclass EMPLOYEE (EmpID, Name, Salary) specialized into:

- STORE_MANAGER (MonthlyBonus, StoreManaged).
- CASHIER (RegisterAssigned, ShiftTiming).

[STORE] (StoreID, Address) 1: N [EMPLOYEE]



[STORE] N: M [PRODUCT] via [INVENTORY]



[CUSTOMER] 1: N [SALE_TRANSACTION] N: 1 [CASHIER]

Relational Schema Translation Highlights

- INVENTORY (StoreID, SKU, StockQuantity) mapped with composite primary key.
- SALE_TRANSACTION (TxnID, TxnDate, TotalAmount, StoreID, EmpID, CustomerID).
- Subclasses mapped via Primary Key / Foreign Key relationship back to superclass PRODUCT table.